Preliminary Requirements Analysis, Technical Roadmap Analysis and timeline planning for Carla2RC

Requirements Analysis

Business Process Description

It is tentatively planned to be implemented in three steps.

Step1. Use Python to implement a control script that outputs the movement command message of the Ackermann model vehicle; realize the script remote control of the RC.

Step2. Synchronize the movement control command of the car in carla when using pygame to control the car to RC; realize the synchronous remote control of RC and carla.

Step3. Build an autonomous driving model in carla, and automatically generate movement control commands through the autonomous driving algorithm and synchronize them to RC; realize carla's synchronous control of the simulated car and RC.

User Cases Description

In step 1, users can use scripts to achieve remote control similar to the front and back + left and right steering of the remote control.

In step 2, users can use pygame to achieve synchronous control of the remote control car and the simulation car in a similar way to the front and back + left and right steering of the remote control.

In step 3, users can set a point in the carla simulation map, and the simulation car and the RC will reach the specified location through automatic driving with the same posture sequence (trajectory).

Functional requirements

As Module division

The following four modules are required.

- **Map**
    - Rasterized city maps as a reference for positioning and navigation
- **Location**
    - RC coordinates are synchronized to the navigation stack
- **Path palnning**

    - Global path planning
        - The shortest path from the current point to the target point that meets the vehicle motion requirements.
    - Path planning
        - Real-time obstacle avoidance.
        - Intercept the global planning fragment and calculate the route that is closest to the actual movement that the car can complete.
        - Pose Tree; Calculate the position states at multiple specific time points to achieve the motion trajectory as much as possible.
    - Movement instructions
- **Communication forwarding**

Non-functional requirements

1. realtime
2. reliable

Technical Roadmap Analysis

Module division

1. Map Module:

- **Function:**
    - Load and manage rasterized city maps.
    - Provide map data to other modules, mainly drivable areas (obstacle information, road network).
- **Input:** Map file (e.g. `.pgm` or `.png` format).
- **Output:** Map data (e.g. ROS `nav_msgs/OccupancyGrid` message).

2. Localization Module:

- **Function:**
    - Get the real-time position information of the RC car (through the vive positioning sensor).
    - Synchronize the coordinate system of the RC car with the coordinate system of the navigation stack.
- **Input:** Sensor data (x,y,z,yaw) of the RC car.
- **Output:** The pose information of the RC car in the coordinate system of the navigation stack (e.g. ROS `geometry_msgs/PoseStamped` message).

3. Path Planning Module:

- **Function:**
    - **Global Path Planning:** Based on the map data and the target point, plan the shortest path from the starting point to the target point and satisfy the vehicle motion constraints.

    - **Local Path Planning:** Based on the sensor information, the global path and the vehicle status, perform real-time obstacle avoidance and generate executable local paths.
    - **Pose Tree:** Calculate the pose states at multiple specific time points to achieve the predetermined motion trajectory as much as possible.
- **Input:**
    - Map data
    - Target point
    - Vehicle status information
    - Sensor data
- **Output:**
    - Global path
    - Local path
    - Pose tree

4. Motion Instruction Module:

- **Function:**
    - Convert the path or pose tree generated by the path planning module into motion instructions (such as speed, steering angle) that can be executed by the RC car.
- **Input:**
    - Path
    - Pose tree
- **Output:**
    - RC car control instructions (such as speed, steering angle).

5. Communication module:

- **Function:**
    - Responsible for the communication between Carla and ROS, RC, and the communication between ROS nodes
    - Communication between VIVE and ROS, mainly coordinates
    - Communication between ROS and Carla includes coordinates, movement instructions, control instructions, etc.
    - Communication between ROS and RC, mainly movement instructions
    - Forward information between RC and Carla through ROS.
- **Input:** Map file (such as `.pgm` or `.png` format).
- **Output:** Map data (such as ROS `nav_msgs/OccupancyGrid` message).

Relationships between modules:

1. The map module provides map data to the path planning module.
2. The positioning module provides the RC car pose information to the path planning module.

3. The path planning module generates global paths, local paths, and pose trees based on map data, target points, vehicle status information, and sensor data.
4. The motion instruction module converts the output of the path planning module into control instructions for the RC car.
5. The communication module is responsible for message forwarding between various modules and different hardware

Data Flow

Key technical difficulties and solutions

Timeline Planning

Specific Task decomposition

step1:(1-2 weeks)

1. Understand the robot's mobile control algorithm and remote control communication
2. Develop RC car control scripts to achieve basic motion control (forward, backward, turn)
3. Test control effects and optimize.

step2:

1. Analyze, design and implementing control prototype of the solution of synchronous control of carla and rc(1-2 weeks)
2. Testing and iterative optimization($\leq$ 1 weeks)

step3:

1. Analyze, design and implement how to provide map and navigation services(1-2 weeks)
2. Navigation stack construction and testing($\leq$ 1 weeks)
3. Simultaneous testing and optimization($\leq$ 1 weeks)

Milestone Event

1. Script remote control test.(ddl:12/08---16/08)
2. Draft architecture design of step2.(ddl:16/08---23/08)
3. Prototype of step2.(ddl:23/08---29/08)
4. Draft architecture design of step3.(ddl:29/08---06/09)
5. Prototype of step3.(ddl:06/09---23/09)